

of course, but concurrent systems programmers do attempt to minimize the amount of *contention* between threads.

As an example, consider a group of threads that are producing data and storing it into a central repository. Every time one of these threads attempts to store its data in the repository, it contends with all of the other threads for this shared resource. A simple solution that can sometimes work is to give each thread its own private repository. The threads can now produce data independently of one another, with no contention. When all of the threads are done producing their output, a central thread can collate the results.

The analog to this approach in the case of the dining philosophers problem would be to give each philosopher two chopsticks from the outset. Doing so would of course remove all concurrency from the problem—without any shared resources, there is no concurrency. In real-world concurrent systems we can't remove *all* sharing of resources, but we can certainly look for ways to *minimize* resource sharing in order to minimize lock contention.

#### 4.8.4 Thread Safety

Generally speaking, a class or functional API is called *thread-safe* when its functions can be safely called by any thread in a multithreaded process. For any one function, thread safety is typically achieved by entering a critical section at the top of the function's body, performing some work, and then leaving the critical section just prior to returning. A class whose functions are all thread-safe is sometimes called a *monitor*. (The term "monitor" is also used to refer to a class that uses condition variables internally to permit clients to sleep while waiting for its protected resources to become available.)

Thread-safety is a convenient feature for a class or interface to offer. But it also imposes overhead that is sometimes unnecessary. For example, this overhead would be wasteful if the interface is being used in a single-threaded program, or if it's being used exclusively by one thread in a multithreaded program.

Thread-safety can also become a problem when an interface function needs to be called reentrantly. For example, if a class provides two thread-safe functions `A()` and `B()`, then these functions cannot call one another because each one independently enters and leaves the critical section. One solution to this problem is to use reentrant locks (see Section 4.9.7.3). Another is to implement "unsafe" versions of the functions in an interface, and then "wrap" each of them in thread-safe variant that simply enters a critical section, calls the "unsafe" function, and then leaves the critical section. That way, we can call the "unsafe" functions internally to the system, but the external interface of the

system remains thread-safe.

In my view, it's a bad idea to attempt to handle concurrent programming by simply creating classes and APIs that are 100% thread-safe. Doing so leads to interfaces that are unnecessarily heavy-weight, and it encourages programmers to ignore the fact that they are working in a concurrent environment. Instead, we should recognize and embrace the presence of concurrency in our software, and aim to design data structures and algorithms that address this concurrency explicitly. The goal should be to produce a software system that minimizes contention and dependencies between threads, and minimizes the use of locks. Achieving a completely lock-free system is a lot of work and is tough to get right. (See Section 4.9.) But striving in the *direction* of lock-freedom (i.e., minimizing the use of locks) is a far better strategy than over-using locks in a futile attempt to create “water-tight” interfaces that allow programmers to be oblivious of concurrency.

## 4.9 Lock-Free Concurrency

Thus far, all of our solutions to the problem of race conditions in a concurrent system have revolved around using *mutex locks* to make critical operations atomic, and possibly leveraging *condition variables* and the kernel's ability to *put threads to sleep* to avoid them wasting valuable CPU cycles in a busy-wait loop. During that discussion, we mentioned that there's another way to avoid race conditions that can potentially be more efficient. That approach is known as *lock-free* concurrency.

Contrary to popular belief, the term “lock-free” doesn't actually refer to the elimination of mutex locks, although that is certainly a component of the approach. In fact, “lock-free” refers to the practice of preventing threads from going to sleep while waiting on a resource to become available. In other words, in lock-free programming we never allow a thread to *block*. So perhaps the term “blocking-free” would have been more descriptive.

Lock-free programming is actually just one of a collection of *non-blocking* concurrent programming techniques. When a thread is blocked, it ceases to make progress. The goal of all of these techniques is to provide guarantees about the *progress* that can be made by the threads in the system, and by the system as a whole. We can organize these techniques into the following categories, listed in order of increasing strength of the progress guarantees they provide:

- *Blocking*. A blocking algorithm is one in which a thread can be put to sleep while waiting for shared resources to become available. A block-

ing algorithm is prone to deadlock, livelock, starvation and priority inversion.

- *Obstruction freedom.* An algorithm is obstruction-free if we can guarantee that a single thread will always complete its work in a bounded number of steps, when all of the other threads in the system are suddenly suspended. The single thread that continues to run is said to be performing a *solo execution* in this scenario, and an obstruction-free algorithm is sometimes called *solo-terminating* because the solo thread eventually terminates while all other threads are suspended. No algorithm that uses a mutex lock or spin lock can be obstruction-free, because if any thread were to be suspended while it is holding a lock, the solo thread might become stuck forever waiting for that lock.
- *Lock freedom.* The technical definition of lock freedom is that in any infinitely-long run of the program, an infinite number of operations will be completed. Intuitively, a lock-free algorithm guarantees that *some* thread in the system can always make progress; in other words, if one thread is arbitrarily suspended, all others can still make progress. This again precludes the use of mutex or spin locks, because if a thread holding a lock is suspended, it can cause other threads to block. A lock-free algorithm is typically transaction-based: A transaction may fail if another thread interrupts it, in which case the transaction is rolled back and retried until it succeeds. This approach avoids deadlock, but it can allow some threads to starve. In other words, certain threads might get stuck in a loop of failing and retrying their transactions indefinitely, while other threads' transactions always succeed.
- *Wait freedom.* A wait-free algorithm provides all the guarantees of lock freedom, but also guarantees starvation freedom. In other words, all threads can make progress, and no thread is allowed to starve indefinitely.

The term “lock-free programming” is sometimes used loosely to refer to *any* algorithm that avoids blocking, but technically speaking the correct term for obstruction-free, lock-free and wait-free algorithms as a whole is “*non-blocking algorithm*.”

The topic of non-blocking algorithms is vast, and is still an open area of research. A complete discussion of the topic would require an entire book of its own. In this chapter, we'll introduce some of the basic principles of lock-free programming. We'll begin by exploring the true causes of data race bugs. Then we'll have a look at how mutexes are actually implemented under the

hood, and learn how to implement our own inexpensive *spin locks*. Finally, we'll present an implementation of a simple lock-free linked list. This discussion ought to be sufficient to give you a flavor for what lock-free data structures and algorithms tend to look like, and should provide a solid jumping off point for further reading and experimentation with lock-free techniques.

#### 4.9.1 Causes of Data Race Bugs

In Section 4.5.3.2, we said that data race bugs occur when a critical operation is interrupted by another critical operation on the same shared data. As it turns out, there are two other ways in which data race bugs can arise, and if we're going to implement our own spin locks or write lock-free algorithms, we'll need to understand them all. Data race bugs can be introduced into a concurrent program:

- via the *interruption* of one critical operation by another,
- by the *instruction reordering* optimizations performed by the compiler and CPU, and
- as a result of hardware-specific *memory ordering semantics*.

Let's break these down a bit further:

- Threads interrupt one another all the time as a result of preemptive multitasking and/or by running them on multiple cores. However, when a *critical* operation is interrupted by another critical operation on the *same* shared data object, a data race bug can occur.
- Optimizing compilers often reorder instructions in an attempt to minimize pipeline stalls. Likewise, the out-of-order execution logic within the CPU can cause instructions to be executed in an order that differs from program order. Instruction reordering is guaranteed not to alter the observable behavior of a single-threaded program. But it *can* alter the way in which two or more threads *cooperate* to share data, thereby introducing bugs into a concurrent program.
- Thanks to aggressive optimizations within a computer's memory controller, the *effects* of a read or write instruction can sometimes become *delayed* relative to other reads and/or writes in the system. As with compiler optimizations and OOO execution, these memory controller optimizations are designed never to alter the observable behavior of a single-threaded program. However, these optimizations *can* change the order of a critical pair of reads and/or writes in a concurrent system,



thereby preventing threads from sharing data in a predictable way. In this book, we'll refer to these kinds of concurrency bugs as *memory ordering bugs*.

In order to guarantee that a critical operation is free from data races, and is therefore atomic, we must ensure that none of these three things can happen.

#### 4.9.2 Implementing Atomicity

First, let's tackle the problem of making a critical operation atomic (i.e., uninterruptible). Before, we waved our hands a bit and said that by wrapping our critical operations in a *mutex* lock/unlock pair, we can magically transform them into atomic operations. But how does a mutex actually work?

##### 4.9.2.1 Atomicity by Disabling Interrupts

To prevent other threads from interrupting our operation, we could try disabling interrupts just prior to performing the operation, making sure to reenabling them after the operation has been completed. That would certainly prevent the kernel from context-switching to another thread in the middle of our atomic operation. However, this approach only works on a single-core machine using preemptive multitasking.

Interrupts are disabled by executing a machine language instruction (such as `cli`, "clear interrupt enable bit," on an Intel x86 architecture). But this kind of instruction only affects the core that executed it. The other cores would continue to run their threads (with interrupts, and therefore preemptive multithreading, still enabled), and those threads could still interrupt our operation. So this approach has only limited applicability in the real world.

##### 4.9.2.2 Atomic Instructions

The term "atomic" suggests the notion of breaking an operation down into smaller and smaller pieces, until we reach a granularity that is indivisible. This idea raises the question: Are there any machine language instructions that are *naturally* atomic? In other words, does the CPU guarantee that some instructions are uninterruptible?

The answer to these questions is "yes," but with a few caveats. There are most certainly some machine language instructions that can *never* be assumed to execute atomically. Other instructions are atomic, but only when operating on certain kinds of data. Some CPUs permit virtually any instruction to be *forced* to execute atomically by specifying a prefix on the instruction in assembly language. (The Intel x86 ISA's `lock` prefix is one example.)

This is good news for concurrent programmers. In fact, it is the existence of these *atomic instructions* that permits us to implement atomicity tools such as mutexes and spin locks, which in turn permit us to make larger-scale operations atomic.

Different CPUs and ISAs provide different sets of atomic instructions, governed by different rules. But we can generalize all atomic instructions as falling into two categories:

- atomic reads and writes, and
- atomic read-modify-write (RMW) instructions.

#### 4.9.2.3 Atomic Reads and Writes

On most CPUs, we can be reasonably certain that a read or write of a four-byte-aligned 32-bit integer will be atomic. That being said, every processor is different, so it's important to always consult your CPU's ISA documentation before relying on the atomicity of any particular instruction.

Some CPUs also support atomic reads and writes of smaller or larger objects, such as single bytes or 64-bit integers, again assuming they are aligned to a multiple of their own size. This is true because on most CPUs, reading and writing an aligned integer whose width in bits is equal to or smaller than the width of a register (or sometimes the width of a cache line) can be performed in a single *memory access cycle*. Because a CPU performs its operation in lock-step with a discrete clock, a memory cycle can't be interrupted, even by another core. As a result, the read or write is effectively atomic.

Misaligned reads and writes usually don't have this atomicity property. This is because in order to read or write a misaligned object, the CPU usually composes two aligned memory accesses. As such, the read or write might be interrupted, and we cannot assume it will be atomic. (See Section 3.3.7.1 for more details on alignment.)

#### 4.9.2.4 Atomic Read-Modify-Write

Atomic reads and writes aren't enough to implement atomic operations in a general sense. In order to implement a locking mechanism like a mutex, we need to be able to read a variable's contents from memory, perform some operation on that variable, and then write the results back to memory, all without being interrupted.

All modern CPUs support concurrency by providing at least one atomic read-modify-write (RMW) instruction. In the following sections, we'll have a look at a few different kinds of atomic RMW instructions. Each has its pros and cons. All of them can be used to implement a mutex or spin lock.

#### 4.9.2.5 Test and Set

The simplest RMW instruction is known as *test-and-set* (TAS). The TAS instruction doesn't actually test and set a value. Rather, it atomically sets a Boolean variable to 1 (true) and returns its *previous* value (so that the value can be tested).

```
// pseudocode for the test-and-set instruction
bool TAS(bool* pLock)
{
    // atomically...
    const bool old = *pLock;
    *pLock = true;
    return old;
}
```

The test-and-set instruction can be used to implement a simple kind of lock called a *spin lock*. Here's some pseudocode illustrating the idea. In this example, we use a hypothetical *compiler intrinsic* called `_tas()` to emit a TAS machine language instruction into our code. Different compilers will provide different intrinsics for this instruction, if the target CPU supports it. For example, under Visual Studio the TAS intrinsic is named `_interlockedbittestandset()`.

```
void SpinLockTAS(bool* pLock)
{
    while (_tas(pLock) == true)
    {
        // someone else has lock -- busy-wait...
        PAUSE();
    }

    // when we get here, we know that we successfully
    // stored a value of true into *pLock AND that it
    // previously contained false, so no one else has
    // the lock -- we're done
}
```

Here, the `PAUSE()` macro indicates the use of a compiler intrinsic, such as Intel's `SSE2 _mm_pause()`, to reduce power consumption during the busy-wait loop. See Section 4.4.6.4 for details on why it's advisable to use a pause instruction inside a busy-wait loop where possible.

It's important to stress here that the above example is intended for illustrative purposes only. It is not 100% correct because it lacks proper memory

fencing. We'll present a complete working example of a spin lock in Section 4.9.7.

#### 4.9.2.6 Exchange

Some ISAs like Intel x86 offer an atomic *exchange* instruction. This instruction swaps the contents of two registers, or a register and a location in memory. On x86, it is atomic by default when exchanging a register with memory (meaning it acts as if the instruction had been preceded with the `lock` prefix).

Here's how to implement a spin lock using an atomic *exchange* instruction. In this example, we're using Visual Studio's `_InterlockedExchange()` *compiler intrinsic* to emit the Intel x86 `xchg` instruction into our code. (Again, note that without proper memory fencing this example is incomplete and won't work reliably. See Section 4.9.7 for a complete implementation.)

```
void SpinLockXCHG(bool* pLock)
{
    bool old = true;
    while (true)
    {
        // emit the xchg instruction for 8-bit words
        _InterlockedExchange8(old, pLock);
        if (!old)
        {
            // if we get back false,
            // then the lock succeeded
            break;
        }
        PAUSE();
    }
}
```

Under Microsoft Visual Studio, all of the “interlocked” functions named with a leading underscore are *compiler intrinsics*—they simply emit the appropriate assembly language instruction directly into your code. The Windows SDK provides a set of similarly-named functions *without* the leading underscore—those functions are implemented in terms of the intrinsic where possible, but they're much more expensive because they involve making a kernel call.

#### 4.9.2.7 Compare and Swap

Some CPUs provide an instruction known as *compare-and-swap* (CAS). This instruction checks the existing value in a memory location, and atomically swaps it with a new value *if and only if* the existing value matches an expected value

provided by the programmer. It returns true if the operation succeeded, meaning that the memory location contained the expected value. It returns false if the operation failed because the location's contents were not as expected.

CAS can operate on values larger than a Boolean. Variants are typically provided at least for 32- and 64-bit integers, although smaller word sizes may also be supported.

The behavior of the CAS instruction is illustrated by the following pseudocode:

```
// pseudocode for compare and swap
bool CAS(int* pValue, int expectedValue, int newValue)
{
    // atomically...
    if (*pValue == expectedValue)
    {
        *pValue = newValue;
        return true;
    }
    return false;
}
```

To implement any atomic read-modify-write operation using CAS, we generally apply the following strategy:

1. Read the old value of the variable we're attempting to update.
2. Modify the value in whatever way we see fit.
3. When writing the result, use the CAS instruction instead of a regular write.
4. Iterate until the CAS succeeds.

The CAS instruction allows us to detect data races, by comparing the value that's actually in the memory location at the time of the write with its value *before* the invocation of our read-modify-write operation. In the absence of a race, the CAS instruction acts just like a write instruction. But if the value in memory changes between the read and the write, we know that some other thread beat us to the punch. In that case, we back off and try again.

Implementing a spin lock via compare-and-swap would look something like this, again using a hypothetical compiler intrinsic called `_cas()` to emit the CAS instruction. (This example once again omits the memory fences that would be required to make it work reliably on all hardware—see Section 4.9.7 for a fully-functional spin lock.)

```

void SpinLockCAS(int* pValue)
{
    const int kLockValue = -1; // 0xFFFFFFFF
    while (!_cas(pValue, 0, kLockValue))
    {
        // must be locked by someone else -- retry
        PAUSE();
    }
}

```

And here's how we'd implement an atomic increment using CAS.

```

void AtomicIncrementCAS(int* pValue)
{
    while (true)
    {
        const int oldValue = *pValue; // atomic read
        const int newValue = oldValue + 1;
        if (_cas(pValue, oldValue, newValue))
        {
            break; // success!
        }
        PAUSE();
    }
}

```

On the Intel x86 ISA, the CAS instruction is called `cmpxchg`, and it can be emitted with Visual Studio's `_InterlockedCompareExchange()` compiler intrinsic.

#### 4.9.2.8 ABA Problem

We should mention that the CAS instruction suffers from an inability to detect one specific kind of data race. Consider an atomic RMW write operation in which the read sees the value A. Before we're able to issue the CAS instruction, another thread preempts us, or runs on another core, and writes two values into the location we're trying to atomically update: First it writes the value B, and then it writes the value A again. When our CAS instruction does finally execute, it won't be able to tell the difference between the first A it read and the A that was written by the other thread. Hence it will "think" that no data race occurred, when in fact one did. This is known as the *ABA problem*.

#### 4.9.2.9 Load Linked/Store Conditional

Some CPUs break the compare-and-swap operation into a *pair* of instructions known as *load linked* and *store conditional* (LL/SC). The *load linked* instruction

reads the value of a memory location atomically, and also stores the address in a special CPU register known as the *link register*. The *store conditional* instruction writes a value into the given address, but only if the address matches the contents of the link register. It returns true if the write succeeded, or false if it failed.

*Any* write operation on the bus (including a store conditional) clears the link register to zero. This means that an LL/SC instruction pair is capable of detecting data races, because if any write occurs between the LL and SC instructions, the SC will fail.

An LL/SC instruction pair is used in much the same way a regular read is paired with a CAS instruction. Specifically, an atomic read-modify-write operation would be implemented using the following strategy:

1. Read the old value of the variable via an LL instruction.
2. Modify the value in whatever way we see fit.
3. Write the result using the SC instruction.
4. Iterate until the SC succeeds.

Here's how we'd implement an atomic increment using LL/SC (using the hypothetical compiler intrinsics `_ll()` and `_sc()` to emit the LL and SC instructions, respectively):

```
void AtomicIncrementLLSC(int* pValue)
{
    while (true)
    {
        const int oldValue = _ll(*pValue);
        const int newValue = oldValue + 1;
        if (_sc(pValue, newValue))
        {
            break; // success!
        }
        PAUSE();
    }
}
```

Because the link register is cleared by *any* bus write, the SC instruction may fail spuriously. But that doesn't affect the correctness of an atomic RMW implemented with LL/SC—it just means that our busy-wait loop might end up executing a few more iterations than we'd expect.

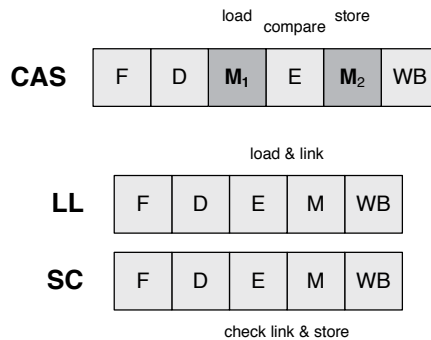


Figure 4.35. The compare and swap (CAS) instruction reads a memory location, performs a comparison, and then conditionally writes to that same location. It therefore requires two memory access stages, making it more difficult to implement within a simple pipelined CPU architecture than the linked/store conditional (LL/SC) instruction pair, each of which requires only a single memory access stage.

#### 4.9.2.10 Advantages of LL/SC over CAS

The LL/SC instruction pair offers two distinct advantages over the single CAS instruction.

First, because the SC instruction fails whenever *any* write is performed on the bus, an LL/SC pair is not prone to the ABA problem.

Second, an LL/SC pair is more pipeline-friendly than a CAS instruction. The simplest pipeline is comprised of five stages: fetch, decode, execute, memory access and register write-back. But a CAS instruction requires two memory access cycles: One to read the memory location so it can be compared to the expected value, and one to write the result if the comparison passes. This means that a pipeline that supports CAS has to include an additional memory access stage that goes unused most of the time. On the other hand, the LL and SC instructions each only require a single memory access cycle, so they fit more naturally into a pipeline with only one memory access stage. A comparison of CAS and LL/SC from a pipelining perspective is shown in Figure 4.35.

#### 4.9.2.11 Strong and Weak Compare-Exchange

The C++11 standard library provides portable functions for performing atomic compare-exchange operations. These may be implemented by a CAS instruction on some target hardware, or by an LL/SC instruction pair on other hardware. Because of the possibility of spurious failures of the store-conditional instruction, C++11 provides two varieties of compare-exchange: strong and weak. Strong compare-exchange “hides” spurious SC failures from the pro-



grammer, while weak compare-exchange does not. Search for “Strong Compare and Exchange Lawrence Crowl” online for a paper on the rationale behind strong and weak compare-exchange functions in C++11.

#### 4.9.2.12 Relative Strength of Atomic RMW Instructions

It’s interesting to note that the TAS instruction is *weaker* than the CAS and LL/SC instructions in terms of achieving *consensus* between multiple threads in a concurrent system. Consensus in this context refers to an agreement between threads about the value of a shared variable (even if some threads in the system experience a failure).

Because the TAS instruction only operates on a Boolean value, it can only solve a problem known as the *wait-free consensus problem* for two concurrent threads. The CAS instruction operates on a 32-bit value, so it can solve this problem for any number of threads.

The topic of wait-free consensus is well beyond our scope here; it’s mostly of interest when building fault-tolerant systems. If you’re interested in fault tolerance, you can read more about the consensus problem by searching for “consensus (computer science)” on Wikipedia.

### 4.9.3 Barriers

Interruptions aren’t the only cause of data race bugs. Compilers and CPUs also conspire to introduce subtle bugs into our concurrent programs by means of the *instruction reordering* optimizations they perform, as described in Section 4.2.5.1.

The cardinal rule of compiler optimizations and out-of-order execution is that their optimizations shall have no visible effects on the behavior of *a single thread*. However, neither the compiler nor the CPU’s control logic has any knowledge of what other threads might be running in the system, or what they might be doing. As a result, the cardinal rule isn’t sufficient to prevent instruction reordering optimizations from introducing bugs into concurrent programs.

The thread synchronization primitives provided by the operating system (mutexes et al.) are carefully crafted to avoid the concurrency bugs that can be caused by instruction reordering optimizations. But now that we’re investigating how mutexes are implemented, let’s take a look at how to avoid these problems manually.

#### 4.9.3.1 How Instruction Reordering Causes Concurrency Bugs

As an example of the kinds of problems instruction reordering can cause in concurrent software, consider again the producer-consumer problem from Section 4.6.3. We've simplified the example, and removed the mutexes so we can expose the bugs that can be introduced by instruction reordering.

```
int32_t g_data = 0;
int32_t g_ready = 0;

void ProducerThread()
{
    // produce some data
    g_data = 42;

    // inform the consumer
    g_ready = 1;
}

void ConsumerThread()
{
    // wait for the data to be ready
    while (!g_ready)
        PAUSE();

    // consume the data
    ASSERT(g_data == 42);
}
```

On a CPU on which aligned reads and writes of 32-bit integers are atomic, this example doesn't actually require mutexes. However, there's nothing to prevent the compiler or the CPU's out-of-order execution logic from reordering the producer's write of 1 into `g_ready` so that it occurs *before* the write of 42 into `g_data`. Likewise, in theory the compiler could reorder the consumer's check that `g_data` is equal to 42 so that it happens *before* the while loop. So even though all of our reads and writes are atomic, this code may not behave reliably.

Instruction reordering really happens at the assembly language level, so it may be a lot more subtle than a reordering of the *statements* in a C/C++ program. For example, the following C/C++ code:

```
A = B + 1;
B = 0;
```

would produce the following Intel x86 assembly code:

```
mov    eax, [B]
add    eax, 1
mov    [A], eax
mov    [B], 0
```

The compiler could very well reorder the instructions as follows, without producing any noticeable effect in a single-threaded execution:

```
mov    eax, [B]
mov    [B], 0    ;; write to B before A!
add    eax, 1
mov    [A], eax
```

If a second thread were waiting for B to become zero before reading the value of A, it would cease to function correctly if this compiler optimization were to be applied.

Jeff Preshing wrote a great blog post on this topic, available at <http://preshing.com/20120625/memory-ordering-at-compile-time/>. (This is where the assembly language example above comes from.) I highly recommend all of Jeff's posts on concurrent programming, so do be sure to check them out.

#### 4.9.3.2 Volatile in C/C++ (and Why It Doesn't Help Us)

How can we prevent the compiler from reordering critical sequences of reads and writes? In C and C++, the `volatile` type qualifier guarantees that consecutive reads or writes of a variable cannot be “optimized away” by the compiler, so this sounds like a promising idea. However, it doesn't work reliably for a number of reasons.

The `volatile` qualifier in C/C++ was really designed to make memory-mapped I/O and signal handlers work reliably. As such, the only guarantee it provides is that the contents of a variable marked `volatile` won't be cached in a register—the variable's value will be re-read directly from memory every time it's accessed. Some compilers do guarantee that instructions will not be reordered across a read or write of a `volatile` variable, but not all of them do, and some only provide this guarantee when targeting certain CPUs, or only when a particular command-line option is passed to the compiler. The C and C++ standards don't require this behavior, so we certainly cannot rely on it when writing portable code. (See <https://msdn.microsoft.com/en-us/magazine/dn973015.aspx> for an in-depth discussion of this topic.)

Moreover, the `volatile` keyword in C/C++ does nothing to prevent the CPU's out-of-order execution logic from reordering the instructions at run-time. And it cannot prevent cache coherency related issues either (see Section 4.9.3). So, at least in C and C++, the `volatile` keyword won't help us to write reliable concurrent software.<sup>8</sup>

#### 4.9.3.3 Compiler Barriers

One reliable way of preventing the compiler from reordering read and write instructions across critical operation boundaries is to explicitly instruct it not to do so. This can be accomplished by inserting a special pseudoinstruction into our code known as a *compiler barrier*.

Different compilers express barriers with different syntax. With GCC, a compiler barrier can be inserted via some inline assembly syntax as shown below; under Microsoft Visual C++, the compiler intrinsic `_ReadWriteBarrier()` has the same effect.

```
int32_t  g_data = 0;
int32_t  g_ready = 0;

void ProducerThread()
{
    // produce some data
    g_data = 42;

    // dear compiler: please don't reorder
    // instructions across this barrier!
    asm volatile("" ::: "memory")

    // inform the consumer
    g_ready = 1;
}

void ConsumerThread()
{
    // wait for the data to be ready
    while (!g_ready)
        PAUSE();

    // dear compiler: please don't reorder
    // instructions across this barrier!
    asm volatile("" ::: "memory")
}
```

---

<sup>8</sup>In some languages including Java and C#, the `volatile` type qualifier *does* guarantee atomicity, and can be used to implement concurrent data structures and algorithms. See Section 4.9.6 for more on this topic.

```
// consume the data
ASSERT(g_data == 42);
}
```

There are other ways to prevent the compiler from reordering instructions. For example, most function calls serve as an implicit compiler barrier. This makes sense, because the compiler doesn't know anything<sup>9</sup> about the side effects of a function call. As such, it can't assume that the state of memory will be the same before and after the call, which means most optimizations aren't permitted across a function call. Some optimizing compilers do make an exception to this rule for inline functions.

Unfortunately, compiler barriers don't prevent the CPU's out-of-order execution logic from reordering instructions at runtime. Some ISAs provide a special instruction for this purpose (e.g., PowerPC's `isync` instruction). In Section 4.9.4.5, we'll learn about a collection of machine language instructions known as *memory fences* which serve as instruction reordering barriers for both the compiler and the CPU, but more importantly also prevent *memory reordering* bugs. So atomic instructions and fences are all we really need to write reliable mutexes, as well as spin locks and other *lock-free* algorithms.

#### 4.9.4 Memory Ordering Semantics

In Section 4.9.1, we said that in addition to the compiler or CPU actually reordering the machine language instructions in our programs, it's possible for read and write instructions to become *effectively reordered* in a concurrent system. Specifically, in a multicore machine with a multilevel memory cache, two or more cores can sometimes *disagree* about the apparent order in which a sequence of read and write instructions occurred, even when the instructions were *actually* executed in the order we intended. Obviously, such disagreements can cause subtle bugs in concurrent software.

These mysterious and vexing problems can only occur on a multicore machine with a multilevel cache. A single CPU core always "sees" the effects of its own read and write instructions in the order they were executed; only when there are two or more cores can disagreements arise. What's more, different CPUs have different memory ordering behavior, meaning that these

---

<sup>9</sup>Function calls only serve as implicit barriers when the compiler is unable to "see" the definition of the function, such as when the function is defined in a separate translation unit. Link time optimizations (LTO) can introduce concurrency bugs by providing a way for the compiler's optimizer to see the definitions of functions it otherwise could not have seen, thereby effectively eliminating these implicit barriers.

strange effects can differ from machine to machine when running the exact same source program!

Thankfully, all is not lost. Every CPU is governed by a strict set of rules known as its *memory ordering semantics*. These rules provide various guarantees about how reads and writes propagate between cores, and they provide programmers with the tools necessary to enforce a particular ordering, when the default semantics are insufficient.

Some CPUs offer only weak guarantees by default, while others provide stronger guarantees and hence require less intervention on the part of the programmer. So, if we can understand how to overcome memory ordering issues on a CPU with the weakest memory ordering semantics, we can be pretty sure those techniques will also work on CPUs with stronger default semantics.

#### 4.9.4.1 Memory Caching Revisited

In order to understand how these mysterious memory reordering effects can occur, we need to look more closely at how a multilevel memory cache works.

In Section 3.5.4, we described in detail how a memory cache avoids the very high memory access latency of main RAM by keeping frequently-used data in the cache. This means that as long as a data object is present in the cache, the CPU will always try to work with that cached copy, rather than reaching out to access the copy in main RAM.

Let's briefly review how this works by considering the following simple (and totally contrived) function:

```
constexpr int      COUNT = 16;
alignas(64) float  g_values[COUNT];
float              g_sum = 0.0f;

void CalculateSum()
{
    g_sum = 0.0f;
    for (int i = 0; i < COUNT; ++i)
    {
        g_sum += g_values[i];
    }
}
```

The first statement sets `g_sum` to zero. Presuming that the contents of `g_sum` aren't already in the L1 cache, the cache line containing it will be read into L1 at this point. Likewise, on the first iteration of the loop, the cache line containing all of the elements of the `g_values` array will be loaded into L1. (They should all fit presuming our cache lines are at least 64 bytes wide, because we aligned

the array to a 64-byte boundary with C++11's `alignas` specifier.) Subsequent iterations will read the copy of `g_values` that resides in the L1 cache, rather than reading them from main RAM.

During each iteration, `g_sum` is updated. The compiler might optimize this by keeping the sum in a register until the end of the loop. But whether or not this optimization is performed, we know that the `g_sum` variable will be written to at some point during this function. When that happens, again the CPU will write to the *copy* of `g_sum` that exists in the L1 cache, rather than writing directly to main RAM.

Eventually, of course, the “master” copy of `g_sum` has to be updated. The memory cache hardware does this automatically by triggering a *write-back* operation that copies the cache line from L1 back to main RAM. But the write-back doesn't usually happen right away; it's typically deferred until the modified variable is read again.<sup>10</sup>

#### 4.9.4.2 Multicore Cache Coherency Protocols

In a multicore machine, memory caching gets a lot more complicated. Figure 4.36 illustrates a simple dual-core machine, in which each core has its own private L1 cache, and the two cores share an L2 cache and a large bank of main RAM. To keep the following discussion as simple as possible, let's ignore the L2 cache and treat it as being roughly equivalent to main RAM.

Suppose that the simplified producer-consumer example shown in Section 4.9.3.3 is running on this dual-core machine. The producer thread runs on Core 1, and the consumer thread runs on Core 2. Let's further assume for the purposes of this discussion that none of the instructions in either thread have been reordered.

```
int32_t  g_data = 0;
int32_t  g_ready = 0;

void ProducerThread() // running on Core 1
{
    g_data = 42;
    // assume no instruction reordering across this line
    g_ready = 1;
}
```

<sup>10</sup>Some memory cache hardware does allow cached writes to *write-through* to main RAM immediately. We can safely ignore write-through caches for the purposes of the present discussion.

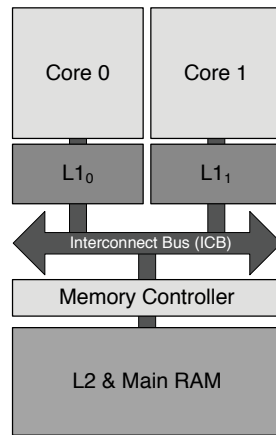


Figure 4.36. A dual core machine with a local L1 cache per core, connected to a memory controller via the interconnect bus (ICB). The memory controller implements a cache coherency protocol such as MESI to ensure that both cores have a consistent view of the contents of memory within the CPU's cache coherency domain.

```

void ConsumerThread() // running on Core 2
{
    while (!g_ready)
        PAUSE();
    // assume no instruction reordering across this line
    ASSERT(g_data == 42);
}
  
```

Now consider what happens when the producer (on Core 1) writes to `g_ready`. In the name of efficiency, this write causes Core 1's L1 cache to be updated, but it won't trigger a *write-back* to main RAM until some time later. This means that for some finite amount of time after the write has occurred, the most up-to-date value of `g_ready` don't exist anywhere but inside Core 1's L1 cache.

Let's say that the consumer (running on Core 2) attempts to read `g_ready` at some time after the producer set it to 1. Like Core 1, Core 2 prefers to read from the cache whenever possible, to avoid the high cost of reading main RAM. Core 2's local L1 cache does not contain a copy of `g_ready`, but Core 1's L1 cache *does*. So ideally Core 2 would like to ask Core 1 for its copy, because that would still be a lot less expensive than getting the data from main RAM. And in this particular case, doing so would also have the distinct advantage of returning the most up-to-date value.

A *cache coherency protocol* is a communication mechanism that permits cores to share data between their local L1 caches in this way. Most CPUs use either the MESI or MOESI protocol.



#### 4.9.4.3 The MESI Protocol

Under the MESI protocol, each cache line can be in one of four states:

- **Modified.** This cache line has been modified (written to) locally.
- **Exclusive.** The main RAM memory block corresponding to this cache line exists *only* in this core's L1 cache—no other core has a copy of it.
- **Shared.** The main RAM memory block corresponding to this cache line exists in *more than one* core's L1 cache, and all cores have an identical copy of it.
- **Invalid.** This cache line no longer contains valid data—the next read will need to obtain the line either from another core's L1 cache, or from main RAM.

The MOESI protocol adds another state named **Owned**, which allows cores to share modified data without writing it back to main RAM first. We'll focus on MESI here in the name of simplicity.

Under the MESI protocol, all cores' L1 caches are connected via a special bus called the *interconnect bus* (ICB). Collectively, the L1 caches, any higher-level caches, and main RAM form what is known as a *cache coherency domain*. The protocol ensures that all cores have a consistent "view" of the data in this domain.

We can get a feel for how the MESI state machine works by returning to our example.

- Let's assume that Core 1 (the producer) first tries to read the current value of `g_ready` for some reason. Presuming that this variable doesn't already exist in any core's L1 cache, the cache line that contains it is loaded into Core 1's L1 cache. The cache line is put into the **Exclusive** state, meaning that no other core has this line.
- Now let's say that Core 2 (the consumer) attempts to read `g_ready`. A *Read* message is sent over the ICB. Core 1 has this cache line, so it responds with a copy of the data. At this point, the cache line is put into the **Shared** state on both cores, indicating that both have an identical copy of the line.
- Next, the producer on Core 1 writes a 1 into `g_ready`. This updates the value in Core 1's L1 cache, and puts its copy of the line into the **Modified** state. An *Invalidate* message is sent across the ICB, causing Core 2's copy of the line to be put into the **Invalid** state. This indicates that Core 2's copy of the line containing `g_ready` is no longer up-to-date.

- The next time that Core 2 (the consumer) tries to read `g_ready`, it finds that its locally-cached copy is **Invalid**. It sends a *Read* message across the ICB and obtains the newly-modified line from Core 1's L1 cache. This causes both cores' cache lines to be put into the **Shared** state once again. It also triggers a *write-back* of the line to main RAM.

A complete discussion of the MESI protocol is beyond our scope, but this example should give you a good feel for how it works to allow multiple cores to share data between their L1 caches while minimizing accesses to main RAM.

#### 4.9.4.4 How MESI Can Go Wrong

Based on the discussion of the MESI protocol in the preceding section, it would seem that the problem of data sharing between L1 caches in a multicore machine has been solved in a watertight way. How, then, can the memory ordering bugs we've hinted at actually happen?

There's a one-word answer to that question: Optimization. On most hardware, the MESI protocol is highly optimized to minimize latency. This means that some operations aren't actually performed immediately when messages are received over the ICB. Instead, they are deferred to save time. As with compiler optimizations and CPU out-of-order execution optimizations, MESI optimizations are carefully crafted so as to be undetectable by a single thread. But, as you might expect, concurrent programs once again get the raw end of this deal.

For example, our producer (running on Core 1) writes 42 into `g_data` and then immediately writes 1 into `g_ready`. Under certain circumstances, optimizations in the MESI protocol can cause the new value of `g_ready` to become visible to other cores within the cache coherency domain *before* the updated value of `g_data` becomes visible. This can happen, for example, if Core 1 already has `g_ready`'s cache line in its local L1 cache, but does *not* have `g_data`'s line yet. This means that the consumer (on Core 2) can potentially see a value of 1 for `g_ready` *before* it sees a value of 42 in `g_data`, resulting in a data race bug.

This state of affairs can be summarized as follows:

Optimizations within a cache coherency protocol can make two read and/or write instructions *appear* to happen, from the point of view of other cores in the system, in an order that is opposite to the order in which the instructions were actually executed.

#### 4.9.4.5 Memory Fences

When the apparent order of two instructions gets reversed by our cache coherency protocol, we say that the first instruction (in program order) has *passed* the second. There are four ways in which one instruction can pass another:

1. A read can pass another read,
2. a read can pass a write,
3. a write can pass another write, or
4. a write can pass a read.

To prevent the memory effects of a read or write instruction passing other reads and/or writes, modern CPUs provide special machine language instructions known as *memory fences*, also known as *memory barriers*.

In theory, a CPU could provide individual fence instructions to prevent each of these four cases from happening. For example, a *ReadRead* fence would only prevent reads from passing other reads, but would not prevent any of the other cases. Also, a fence instruction could be unidirectional or bidirectional. A one-way fence would guarantee that no reads or writes that precede it in program order can end up having an effect after it, but not vice-versa. A bidirectional fence, on the other hand, would prevent “leakage” of memory effects in either direction across the fence. Theoretically, then, we could imagine a CPU that provides twelve distinct fence instructions—a bidirectional, forward, and reverse variant of each of the four basic fence types listed above.

Thankfully real CPUs don’t usually provide all twelve kinds of fences. Instead, an ISA typically specifies a handful of fence instructions which serve as combinations of these theoretical fence types.

The strongest kind of fence is called a *full fence*. It ensures that all reads and writes occurring before the fence in program order will never appear to have occurred after it, and likewise that all reads and writes occurring after it will never appear to have happened before it. In other words, a full fence is a two-way barrier that affects both reads and writes.

A full fence is actually very expensive to realize in hardware. CPU designers don’t like forcing programmers to use an expensive construct when a cheaper one will do, so most CPUs provide a variety of less-expensive fence instructions which provide weaker guarantees than those provided by a full fence.

All fence instructions have two very useful side-effects:

1. They serve as compiler barriers, and

2. they prevent the CPU's out-of-order logic from reordering instructions across the fence.

This means that when we use a fence to prevent the memory ordering bugs that are caused by our CPU's cache coherency protocol, it also serves to prevent instruction reordering. So atomic instructions and memory fences are all we really need to write reliable mutexes and spin locks, and to write other *lock-free* algorithms as well.

#### 4.9.4.6 Acquire and Release Semantics

No matter what the specific fence instructions look like under a particular ISA, we can reason about their effects by thinking in terms of the *semantics* they provide—in other words, the guarantees they enforce about the behavior of reads and writes in the system.

Memory ordering semantics are really properties of read or write instructions, not properties of the fences themselves. The fences merely provide a way for programmers to ensure that a read or write instruction has a particular memory ordering semantic. There are really only three memory ordering semantics we typically need to worry about:

- *Release* semantics. This semantic guarantees that a *write* to shared memory can never be passed by any other read or write that *precedes* it in program order. When this semantic is applied to a shared write, we call it a *write-release*. This semantic operates in the *forward* direction only—it says nothing about preventing memory operations that occur after the write-release from appearing to happen before it.
- *Acquire* semantics. This semantic guarantees that a *read* from shared memory can never be passed by any other read or write that occurs *after* it in program order. When this semantic is applied to a shared read, we call it a *read-acquire*. This semantic operates in the *reverse* direction only—it does nothing to prevent memory operations that occur before the read-acquire from having their effects seen after it.
- *Full fence* semantics. This bidirectional semantic ensures that all memory operations appear to occur in program order across the boundary created by a fence instruction in the code. No read or write that occurs before the fence in program order can appear to have occurred after the fence, and likewise no read or write that is after the fence in program order can appear to have occurred before it.

The individual fence instructions provided by any particular ISA typically provide at least one of these three memory ordering semantics. The details of how each fence instruction actually provides these semantic guarantees are CPU-specific, and for the most part as concurrent programmers, we don't care. As long as we can express the concept of write-release, read-acquire and full fence in our source code, we should be able to write a reliable spin lock or code other lock-free algorithms.

#### 4.9.4.7 When to Use Acquire and Release Semantics

A *write-release* is most often used in a producer scenario—in which a thread performs two consecutive writes (e.g., writing to `g_data` and then `g_ready`), and we need to ensure that all other threads will see the two writes in the correct order. We can enforce this ordering by making the second of these two writes a write-release. To implement this, a fence instruction that provides *release semantics* is placed *before* the write-release instruction. Technically speaking, when a core executes a fence instruction with release semantics, it waits until all prior writes have been fully committed to memory within the cache coherency domain before executing the second write (the write-release).

A *read-acquire* is typically used in a consumer scenario—in which a thread performs two consecutive reads in which the second is conditional on the first (e.g., only reading `g_data` after a read of the flag `g_ready` comes back `true`). We enforce this ordering by making sure that the first read is a read-acquire. To implement this, a fence instruction that provides *acquire semantics* is placed *after* the read-acquire instruction. Technically speaking, when a core executes a fence instruction with acquire semantics, it waits until all writes from other cores have been fully flushed into the cache coherency domain before it continues on to execute the second read. This ensures that the second read will never appear to have occurred before the read-acquire.

Here's our producer-consumer example again, in full lock-free glory, using acquire and release fences to impose the necessary memory ordering semantics:

```
int32_t  g_data = 0;
int32_t  g_ready = 0;

void ProducerThread() // running on Core 1
{
    g_data = 42;

    // make the write to g_ready into a write-release
    // by placing a release fence *before* it
    RELEASE_FENCE();
```

```

    g_ready = 1;
}

void ConsumerThread() // running on Core 2
{
    // make the read of g_ready into a read-acquire
    // by placing an acquire fence *after* it
    while (!g_ready)
        PAUSE();
    ACQUIRE_FENCE();

    // we can now read g_data safely...
    ASSERT(g_data == 42);
}

```

For an excellent in-depth presentation of exactly why acquire and release fences are required under the MESI cache coherency protocol, see <http://www.swedishcoding.com/2017/11/10/multi-core-programming-and-cache-coherency/>.

#### 4.9.4.8 CPU Memory Models

We mentioned in Section 4.9.4 that some CPUs provide stronger memory order semantics by default than others. On a CPU with strong memory semantics, read and/or write instructions behave like a certain kind of fence by default, without the programmer having to specify a fence instruction explicitly. For example, the DEC Alpha has notoriously weak semantics by default, requiring careful fencing in almost all situations. At the other end of the spectrum, an Intel x86 CPU has quite strong memory ordering semantics by default. For a great discussion of weak and strong memory ordering, see <http://preshing.com/20120930/weak-vs-strong-memory-models/>.

#### 4.9.4.9 Fence Instructions on Real CPUs

Now that we understand the theory behind memory ordering semantics, let's take a very brief look at memory fence instructions on some real CPUs.

The Intel x86 ISA specifies three fence instructions: `sfence` provides release semantics, `lfence` provides acquire semantics, and `mfence` acts as a full fence. Certain x86 instructions may also be prefixed by a `lock` modifier to make them behave atomically, and to provide a memory fence prior to execution of the instruction. The x86 ISA is strongly ordered by default, meaning that fences aren't actually required in many cases where they would be on CPUs with weaker default ordering semantics. But there are some cases in which these fence instructions *are* required. For an example, see the post

entitled, “Who ordered memory fences on an x86?” by Bartosz Milewski (<https://bit.ly/2HuXpfo>).

The PowerPC ISA is quite weakly ordered, so explicit fence instructions are usually required to ensure correct semantics. The PowerPC makes a distinction between reads and writes to memory versus reads and writes to I/O devices, and hence it offers a variety of fence instructions that differ primarily in how they handle memory versus I/O. A full fence on PowerPC is provided by the `sync` instruction, but there’s also a “lightweight” fence called `lwsync`, a fence for I/O operations called `eieio` (ensure in-order execution of I/O), and even a pure instruction reordering barrier `isync` that does not provide any memory ordering semantics. You can read more about the PowerPC’s fence instructions here: <https://www.ibm.com/developerworks/systems/articles/powerpc.html>.

The ARM ISA provides a pure instruction reordering barrier called `isb`, two full memory fence instructions `dmb` and `dsb`, a one-way read-acquire instruction `ldar` and a one-way write-release instruction `stlr`. Interestingly, this ISA rolls acquire and release semantics into the read and write instructions themselves, rather than as separate fence instructions. For more information, see <http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.den0024a/CJAIAJFI.html>.

#### 4.9.5 Atomic Variables

Using atomic instructions and memory fences directly can be tedious and error-prone, not to mention being totally non-portable. Thankfully, as of C++11, the class template `std::atomic<T>` allows virtually any data type to be converted into an *atomic variable*. (A specialized class called `std::atomic_flag` encapsulates an atomic Boolean variable.) In addition to atomicity, the `std::atomic` family of templates provides its variables with “full fence” memory ordering semantics by default (although weaker semantics can be specified if desired). This frees us to write lock-free code without having to worry about any of the three causes of data race bugs.

Using these facilities, our producer-consumer example can be written as follows:

```
std::atomic<float> g_data;
std::atomic_flag g_ready = false;

void ProducerThread()
{
    // produce some data
    g_data = 42;
```

```

        // inform the consumer
        g_ready = true;
    }

    void ConsumerThread()
    {
        // wait for the data to be ready
        while (!g_ready)
            PAUSE();

        // consume the data
        ASSERT(g_data == 42);
    }

```

Notice that this code looks nearly identical to the erroneous code we first presented when talking about race conditions in Section 4.5.3. By simply wrapping our variables in `std::atomic`, we've converted a concurrent program that is prone to data race bugs into one that is race-free.

Under the hood, the implementations of `std::atomic<T>` and `std::atomic_flag` are, of course, complex. The standard C++ library has to be portable, so its implementation makes use of whichever atomic and barrier machine language instructions happen to be available on the target platform. What's more, the `std::atomic<T>` template can wrap any type imaginable, but of course CPUs don't provide atomic instructions for manipulating arbitrarily-sized data objects. As such, the `std::atomic<T>` template must be specialized by size: When the template is applied to a 32- or 64-bit type, it can be implemented without locks, using atomic machine language instructions directly. But when it is applied to larger types, mutex locks are used to ensure correct atomic behavior. (You can call `is_lock_free()` on any atomic variable to find out whether its implementation really is lock-free on your target hardware.)

#### 4.9.5.1 C++ Memory Order

By default, C++ atomic variables make use of full memory barriers, to ensure that they will work correctly in all possible use cases. However, it is possible to *relax* these guarantees, by passing a *memory order semantic* (an optional argument of type `std::memory_order`) to functions that manipulate atomic variables. The documentation regarding `std::memory_order` is pretty confusing, so let's demystify it. Here are the possible memory order settings, and what they mean:

1. *Relaxed*. An atomic operation performed with relaxed memory order



semantics guarantees *only* atomicity. No barrier or fence is used.

2. *Consume*. A read performed with consume semantics guarantees that no other read or write *within the same thread* can be reordered before this read. In other words, this semantic only serves to prevent *compiler optimizations* and *out-of-order execution* from reordering the instructions—it does nothing to ensure any particular memory ordering semantics within the cache coherency domain. It is typically implemented with an instruction reordering barrier like the PowerPC’s `isync` instruction.
3. *Release*. A write performed with *release* semantics guarantees that no other read or write in this thread can be reordered after it, and the write is guaranteed to be visible by other threads reading the same address. It employs a *release fence* in the CPU’s cache coherency domain to accomplish this.
4. *Acquire*. A read performed with *acquire* semantics guarantees *consume* semantics, and in addition it guarantees that writes to the same address by other threads will be visible to this thread. It does this via an *acquire fence* in the CPU’s cache coherency domain.
5. *Acquire/Release*. This semantic (the default) is the safest, because it applies a *full* memory fence.

It’s important to realize that using a memory ordering specifier doesn’t guarantee that a particular semantic will actually be used on every platform. All it does is to guarantee that the semantics will be *at least* that strong—a stronger semantic might be employed on some target hardware. For example, on Intel x86, the relaxed memory order isn’t possible because the CPU’s default memory ordering semantics are relatively strong. On an Intel CPU, any request for a relaxed read operation will actually end up having *acquire* semantics.

Using these memory order specifiers requires switching from `std::atomic`’s overloaded assignment and cast operators to explicit calls to `store()` and `load()`. Here’s our simple producer-consumer example again, this time using `std::memory_order` specifiers to provide our release and acquire barriers:

```
std::atomic<float> g_data;
std::atomic<bool> g_ready = false;

void ProducerThread()
{
    // produce some data
```

```

    g_data.store(42, std::memory_order_relaxed);

    // inform the consumer
    g_ready.store(true, std::memory_order_release);
}

void ConsumerThread()
{
    // wait for the data to be ready
    while (!g_ready.load(std::memory_order_acquire))
        PAUSE();

    // consume the data
    ASSERT(g_data.load(std::memory_order_relaxed) == 42);
}

```

It’s important to remember the 80/20 rule when using “relaxed” memory ordering semantics like this. These semantics are easy to get wrong, so you should probably only use non-default memory ordering with `std::atomic` when you can prove, via profiling, that the performance improvement is really necessary—and that changing your code to use explicit memory ordering semantics does, in fact, produce the benefits you expect!

A complete discussion of how to use memory ordering semantics in C++11 is beyond our scope, but you can read more by searching online for the article entitled, “Implementing Scalable Atomic Locks for Multi-Core Intel® EM64T and IA32 Architectures” by Michael Chynoweth. This forum discussion offers some interesting insights as well, and illustrates just how complex this kind of programming can get: <https://groups.google.com/forum/#!topic/boost-developers-archive/QLrat5ASrnM>.

#### 4.9.6 Concurrency in Interpreted Programming Languages

Thus far we’ve only discussed concurrency in the context of compiled languages like C and C++, and in assembly language. These languages compile or assemble down to raw machine code that is executed directly by the CPU. As such, atomic operations and locks must be implemented with the aid of special machine language instructions that provide atomic operations and cache coherent memory barriers, with additional help from the kernel (which makes sure threads are put to sleep and awoken appropriately) and the compiler (which respects barrier instructions when optimizing the code).

The story is somewhat different for interpreted programming languages like Java and C#. Programs written in these languages execute in the context of a *virtual machine* (VM): Java programs run inside the Java Virtual Machine

(JVM), and C# programs run within the context of the Common Language Runtime (CLR). A virtual machine is essentially a software emulation of a CPU, reading bytecoded instructions one by one and executing them. A VM also acts a bit like an emulated operating system kernel: It provides its own notion of “threads” (comprised of bytecoded instructions), and it handles all of the details of scheduling those threads itself. Because the operation of a VM is implemented entirely in software, an interpreted language like Java or C# can provide powerful concurrent synchronization facilities that are not as constrained by the hardware as they would be in a compiled language like C or C++.

One example of this principle in action is the `volatile` type qualifier. We said in Section 4.6 that in C/C++ a `volatile` variable is not atomic. However, in Java and C#, the `volatile` type qualifier *does* guarantee atomicity. Operations on volatile variables in these languages cannot be optimized, and they cannot be interrupted by another thread. Moreover, all reads of a volatile variable in Java and C# are effectively performed directly from main memory rather than from the cache, and likewise all writes are effectively written to main RAM rather than to the cache. All of these guarantees can be provided in part because the virtual machine has full control over the execution of the bytecoded instruction streams that comprise each app.

A full discussion of the atomicity and thread synchronization facilities provided by interpreted languages like C# and Java is well beyond the scope of this book. But now that you have a solid understanding of the principles behind atomicity at the lowest levels, understanding these facilities in other higher-level languages should be a snap. For further reading, the following websites are a good start:

- C#: Search for “Parallel Processing and Concurrency in the .NET Framework” on <https://docs.microsoft.com>.
- Java: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>.

#### 4.9.7 Spin Locks

When discussing atomic machine language instructions in Section 4.9.2.2, we presented some code snippets illustrating how a spin lock might be implemented using each of those instructions. But because of instruction reordering and memory ordering semantics, those examples weren’t 100% correct. In this section, we’ll present an industrial-strength spin lock, and then explore a few useful variants.

#### 4.9.7.1 Basic Spin Lock

A spin lock can be implemented using a `std::atomic_flag`, either wrapped in a C++ class or accessed via a simple functional API. The spin lock is acquired by using a TAS instruction to atomically set the flag to `true`, retrying in a while loop until the TAS succeeds. It is unlocked by atomically writing a value of `false` into the flag.

When acquiring a spin lock, it's important to use *read-acquire* memory ordering semantics to read the lock's current contents as part of the TAS operation. This fence guards against the rare scenario in which the lock is observed as having been released when in reality some other thread hasn't *completely* exited its critical section yet. In C++11, this can be accomplished by passing `std::memory_order_acquire` to the `test_and_set()` call. In raw assembly language, we'd place an acquire fence instruction after the TAS instruction.

When releasing a spin lock, it's likewise important to use *write-release* semantics to ensure that all writes performed *after* the call to `Unlock()` aren't observed by other threads as if they had happened *before* the lock was released.

Here's a complete implementation of a TAS-based spin lock, using correct and minimal memory ordering semantics:

```
class SpinLock
{
    std::atomic_flag m_atomic;

public:
    SpinLock() : m_atomic(false) { }

    bool TryAcquire()
    {
        // use an acquire fence to ensure all subsequent
        // reads by this thread will be valid
        bool alreadyLocked = m_atomic.test_and_set(
            std::memory_order_acquire);

        return !alreadyLocked;
    }

    void Acquire()
    {
        // spin until successful acquire
        while (!TryAcquire())
        {
```

```

        // reduce power consumption on Intel CPUs
        // (can substitute with std::this_thread::yield()
        // on platforms that don't support CPU pause, if
        // thread contention is expected to be high)
        PAUSE();
    }
}

void Release()
{
    // use release semantics to ensure that all prior
    // writes have been fully committed before we unlock
    m_atomic.clear(std::memory_order_release);
}
};

```

#### 4.9.7.2 Scoped Locks

It's often inconvenient and error-prone to manually unlock a mutex or spin lock, especially when the function using the lock has multiple return sites. In C++ we can use a simple wrapper class called a *scoped lock* to ensure that a lock is automatically released when a particular scope is exited. It works by simply acquiring the lock in the constructor, and releasing it in the destructor.

```

template<class LOCK>
class ScopedLock
{
    typedef LOCK lock_t;
    lock_t* m_pLock;

public:
    explicit ScopedLock(lock_t& lock) : m_pLock(&lock)
    {
        m_pLock->Acquire();
    }

    ~ScopedLock()
    {
        m_pLock->Release();
    }
};

```

This scoped lock class can be used with any spin lock or mutex that has a conformant interface (i.e., any lock class that supports the functions `Acquire()` and `Release()`). Here's how it might be used:

```

SpinLock g_lock;

int ThreadSafeFunction()
{
    // the scoped lock acts like a "janitor"
    // because it cleans up for us!
    ScopedLock<decltype(g_lock)> janitor(g_lock);

    // do some work...

    if (SomethingWentWrong())
    {
        // lock will be released here
        return -1;
    }

    // so some more work...

    // lock will also be released here
    return 0;
}

```

#### 4.9.7.3 Reentrant Locks

A vanilla spin lock will cause a thread to deadlock if it ever tries to reacquire a lock that it already holds. This can happen whenever two or more *thread-safe* functions attempt to call one another reentrantly from within the same thread. For example, given two functions:

```

SpinLock g_lock;

void A()
{
    ScopedLock<decltype(g_lock)> janitor(g_lock);

    // do some work...
}

void B()
{
    ScopedLock<decltype(g_lock)> janitor(g_lock);

    // do some work...

    // make a call to A() while holding the lock
    A(); // deadlock!
}

```

```

        // do some more work...
    }

```

We can relax this reentrancy restriction if we can arrange for our spin lock class to cache the id of the thread that has locked it. That way, the lock can “know” the difference between a thread trying to reacquire its own lock (which we wish to allow) versus a thread trying to grab a lock that’s already held by a different thread (which should cause the caller to wait). To make sure that calls to `Acquire()` and `Release()` are done in matching pairs, we’ll also want to add a reference count to the class. Here’s a functional implementation, using appropriate memory fencing:

```

class ReentrantLock32
{
    std::atomic<std::size_t> m_atomic;
    std::int32_t m_refCount;

public:
    ReentrantLock32() : m_atomic(0), m_refCount(0) { }

    void Acquire()
    {
        std::hash<std::thread::id> hasher;
        std::size_t tid = hasher(std::this_thread::get_id());

        // if this thread doesn't already hold the lock...
        if (m_atomic.load(std::memory_order_relaxed) != tid)
        {
            // ... spin wait until we do hold it
            std::size_t unlockValue = 0;
            while (!m_atomic.compare_exchange_weak(
                unlockValue,
                tid,
                std::memory_order_relaxed, // fence below!
                std::memory_order_relaxed))
            {
                unlockValue = 0;
                PAUSE();
            }
        }

        // increment reference count so we can verify that
        // Acquire() and Release() are called in pairs
        ++m_refCount;
    }
}

```

```

        // use an acquire fence to ensure all subsequent
        // reads by this thread will be valid
        std::atomic_thread_fence(std::memory_order_acquire);
    }

void Release()
{
    // use release semantics to ensure that all prior
    // writes have been fully committed before we unlock
    std::atomic_thread_fence(std::memory_order_release);

    std::hash<std::thread::id> hasher;
    std::size_t tid = hasher(std::this_thread::get_id());
    std::size_t actual = m_atomic.load(std::memory_order_relaxed);
    assert(actual == tid);

    --m_refCount;
    if (m_refCount == 0)
    {
        // release lock, which is safe because we own it
        m_atomic.store(0, std::memory_order_relaxed);
    }
}

bool TryAcquire()
{
    std::hash<std::thread::id> hasher;
    std::size_t tid = hasher(std::this_thread::get_id());

    bool acquired = false;

    if (m_atomic.load(std::memory_order_relaxed) == tid)
    {
        acquired = true;
    }
    else
    {
        std::size_t unlockValue = 0;
        acquired = m_atomic.compare_exchange_strong(
            unlockValue,
            tid,
            std::memory_order_relaxed, // fence below!
            std::memory_order_relaxed);
    }
    if (acquired)
    {
        ++m_refCount;
    }
}

```



```

        std::atomic_thread_fence(
            std::memory_order_acquire);
    }
    return acquired;
}
};

```

#### 4.9.7.4 Readers-Writer Locks

In a system in which multiple threads can read and write a shared data object, we could control access to the object using a mutex or a spin lock. However, multiple threads should be allowed to *read* the shared object concurrently. It's only when the shared object is being *mutated* that we need to ensure mutual exclusivity. What we'd like is a kind of lock that allows any number of readers to acquire it concurrently. Whenever a writer thread attempts to acquire the lock, it should wait until all readers are finished, and then it should acquire the lock in a special "exclusive" mode that prevents any other readers or writers from gaining access until it has completed its mutation of the shared object. This kind of lock is called a *readers-writer lock* (also known as a *shared-exclusive lock* or a *push lock*).

We can implement a readers-writer lock in a manner similar to how we implemented our reentrant lock. However, instead of storing the thread id in the atomic variable, we'll store a reference count indicating how many readers currently hold the lock. Each time a reader acquires the lock, the count is incremented; each time a reader releases the lock, the count is decremented.

But how, then, can we also provide an "exclusive" lock mode for the writer? All we need to do is to reserve one (very high) reference count value and use it to denote that a writer currently holds the lock. If our reference count is an unsigned 32-bit integer, the value 0xFFFFFFFFU could do nicely as the reserved value. Even easier, we can simply reserve the most-significant bit, meaning that reference counts from 0 to 0x7FFFFFFFU represent reader locks, and the reserved value 0x80000000U represents a writer lock (with no other values being valid).

A readers-writer lock suffers from starvation problems: A writer that holds the lock too long can cause all of the readers to starve, and likewise a lot of readers can cause the writers to starve. A *sequential lock* is one possible alternative that tackles the starvation issue (see <https://en.wikipedia.org/wiki/Seqlock> for details). Check out <https://lwn.net/Articles/262464> for a description of yet another interesting locking technique, used in the Linux kernel, that supports multiple concurrent readers and writers, called *read-copy-update* (RCU).

We'll leave the implementation of a readers-writer lock up to you as an exercise! However, if you'd like to compare notes, you can find a fully-functional implementation on this book's website ([www.gameenginebook.com](http://www.gameenginebook.com)).

#### 4.9.7.5 Lock-Not-Needed Assertions

No matter how you slice it, locks are expensive. Mutexes are expensive even in the absence of contention. In a low-contention scenario, spin locks are relatively cheap, but they still introduce a non-zero cost into any piece of software.

It's often the case that the programmer knows *a priori* that a lock isn't required. In a game engine, for example, each iteration of the game loop is usually performed in distinct stages. If a shared data structure is accessed exclusively by a single thread early in the frame, and that same data is accessed again by a single thread later in the frame, then we don't actually need a lock. Yes, in theory those two threads could overlap, and if they were to do so a lock would definitely be needed. But in practice, given the way our game loop is structured, we might know that such overlap can never occur.

In this kind of situation, we have a few choices. We could put the locks in, just in case. That way, if someone rearranges the order in which things are done during the frame, and ends up making these threads overlap, we'd be covered. Another option is to just ignore the possibility of overlap and not lock anything.

There is a third option which I find more appealing in such scenarios: We can *assert* that a lock isn't required. This has two benefits. First, it can be done very cheaply, and in fact the assertions can be stripped prior to shipping your game. Second, it automatically detects problems if our assumptions about the overlap of the threads proves to be incorrect—or if the assumptions are broken later on by a refactor of the code. There's no standardized name for this kind of assertion, so we'll call them *lock-not-needed assertions* in this book.

So how do we detect that a lock is needed? One way would be to use an atomic Boolean variable, complete with proper memory fencing, and use it like a mutex. Except that instead of actually acquiring a mutex lock, we would simply assert that the Boolean is false, and then set it to true atomically. And instead of releasing a lock, we assert that our Boolean is true, and then set it to false atomically. This approach would work, but it would be just as expensive as an uncontended spin lock. We can do better.

The "trick" is to realize that we only care about *detecting* overlaps between critical operations on a shared object. And that detection needn't be 100% reliable. A 90% hit rate is probably just fine. If two critical operations ever *do* overlap, there may be times when we fail to detect it. But if your game is being run multiple times a day, every day, by a team of 100 or more developers,

plus a QA department consisting of at least another 10 or 20 people, you can be pretty sure *someone* will detect the problem if one exists.

So, instead of an *atomic* Boolean, we'll just use a *volatile* Boolean. As we've stated, the *volatile* keyword doesn't do much to prevent concurrent race bugs. But it does guarantee that reads and writes of the Boolean won't be optimized away, and that's really all we need. We'll get a reasonably good detection rate, and the test is dirt cheap.

```
class UnnecessaryLock
{
    volatile bool    m_locked;

public:
    void Acquire()
    {
        // assert no one already has the lock
        assert(!m_locked);

        // now lock (so we can detect overlapping
        // critical operations if they happen)
        m_locked = true;
    }
    void Release()
    {
        // assert correct usage (that Release()
        // is only called after Acquire())
        assert(m_locked);

        // unlock
        m_locked = false;
    }
};

#if ASSERTIONS_ENABLED
#define BEGIN_ASSERT_LOCK_NOT_NECESSARY(L) (L).Acquire()
#define END_ASSERT_LOCK_NOT_NECESSARY(L)   (L).Release()
#else
#define BEGIN_ASSERT_LOCK_NOT_NECESSARY(L)
#define END_ASSERT_LOCK_NOT_NECESSARY(L)
#endif

// Example usage...

UnnecessaryLock g_lock;
```

```

void EveryCriticalOperation()
{
    BEGIN_ASSERT_LOCK_NOT_NECESSARY(g_lock);

    printf("perform critical op...\n");

    END_ASSERT_LOCK_NOT_NECESSARY(g_lock);
}

```

We could also wrap the locks in a janitor (see Section 3.1.1.6), like this:

```

class UnnecessaryLockJanitor
{
    UnnecessaryLock* m_pLock;
public:
    explicit
    UnnecessaryLockJanitor(UnnecessaryLock& lock)
        : m_pLock(&lock) { m_pLock->Acquire(); }
    ~UnnecessaryLockJanitor() { m_pLock->Release(); }
};

#ifdef ASSERTIONS_ENABLED
#define ASSERT_LOCK_NOT_NECESSARY(J,L) \
    UnnecessaryLockJanitor J(L)
#else
#define ASSERT_LOCK_NOT_NECESSARY(J,L)
#endif

// Example usage...

UnnecessaryLock g_lock;

void EveryCriticalOperation()
{
    ASSERT_LOCK_NOT_NECESSARY(janitor, g_lock);

    printf("perform critical op...\n");
}

```

We implemented this at Naughty Dog and it successfully caught a number of cases of critical operations overlapping when the programmers had assumed they never could do so. So this little gem is tried and true.

#### 4.9.8 Lock-Free Transactions

This is supposed to be a section on lock-free programming, but thus far we've spent all of our time on writing spin locks! Perhaps counterintuitively, the

act of writing a spin lock is an example of lock-free programming, from the perspective of the implementation of the spin lock itself. We've also learned a lot about atomic instructions, compiler barriers and memory fences along the way. So this has been a useful exercise. However, we haven't really explored the principles of lock-free programming per se; for that purpose it will be instructive to look at an example other than a spin lock. The topic of lock-free and non-blocking algorithms is huge. It really deserves its own book, so we won't attempt to cover the topic in depth here. But we can at least get a feel for how lock-free approaches usually work.

The goal of lock-free programming is of course to avoid taking locks that will either put a thread to sleep, or cause it to get caught up in a busy-wait loop inside a spin lock. To perform a critical operation in a lock-free manner, we need to think of each such operation as a *transaction* that can either succeed in its entirety, or fail in its entirety. If it fails, the transaction is simply retried until it does succeed.

To implement any transaction, no matter how complex, we perform the majority of the work *locally* (i.e., using data that's visible only to the current thread, rather than operating directly on the shared data). When all of our ducks are in a row and the transaction is ready to *commit*, we execute a *single atomic instruction*, such as CAS or LL/SC. If this atomic instruction succeeds, we have successfully "published" our transaction globally—it becomes a permanent part of the shared data structure on which we're operating. But if the atomic instruction fails, that means some *other* thread was attempting to commit a transaction at the same time we were.

This fail-and-retry approach works because whenever one thread fails to commit its transaction, we know that it was because some *other* thread managed to *succeed*. As a result, one thread in the system is always making forward progress (just maybe not us). And that is the definition of *lock-free*.

#### 4.9.9 A Lock-Free Linked List

As a concrete example, let's look at a simple lock-free singly-linked list. The only operation we'll support in this discussion is `push_front()`.

To prepare the transaction, we allocate the new `Node` and populate it with data. We also set its next pointer to point to whichever node is currently at the head of the linked list. The transaction is now ready to commit atomically.

The commit itself consists of a call to `compare_exchange_weak()` on the head pointer, which we declared as an atomic pointer to a `Node`. If this call succeeds, we've inserted our new node at the head of the linked list and we're done. But if it fails, we need to retry. This involves re-initializing our

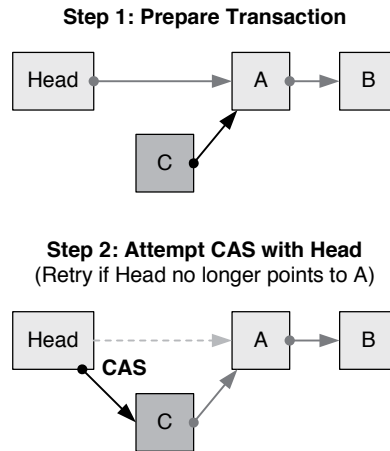


Figure 4.37. A lock-free implementation of insertion at the head of a singly-linked list. Top: The transaction is prepared by setting the next pointer of the new node to point to the current head of the list. Bottom: The transaction is committed by using an atomic CAS operation to swap the head pointer with a pointer to the new node. If the CAS fails, we return to the top and try again until it succeeds.

new node’s next pointer to point to what is now potentially a *brand new* head node (presumably inserted by another thread—this is perhaps why we failed in the first place). This two-stage process is illustrated in Figure 4.37.

In the code below, you won’t see an explicit re-initialization of the node’s next pointer. That’s because `compare_exchange_weak()` does the re-initialization step for us. (How convenient!) Here’s what the code would look like:

```
template< class T >
class SList
{
    struct Node
    {
        T      m_data;
        Node*  m_pNext;
    };
    std::atomic< Node* > m_head { nullptr };

public:
    void push_front(T data)
    {
        // prep the transaction locally
        auto pNode = new Node();
        pNode->m_data = data;
```

```

        pNode->m_pNext = m_head;

        // commit the transaction atomically
        // (retrying until it succeeds)
        while (!m_head.compare_exchange_weak(
            pNode->m_pNext, pNode))
        { }
    }
};

```

#### 4.9.10 Further Reading on Lock-Free Programming

Concurrency is a vast and profound topic, and in this chapter we've only just scratched the surface. As always, the goal of this book is merely to build awareness and serve as a jumping-off point for further learning.

- For a complete discussion of implementing a lock-free singly-linked list, check out Herb Sutter's talk at CppCon 2014, which is where the example above came from. The talk is available on YouTube in two parts:
  - <https://www.youtube.com/watch?v=c1gO9aB9nbs>, and
  - <https://www.youtube.com/watch?v=CmxkPChOcvw>.
- This lecture by Geoff Langdale of CMU provides a great overview: [https://www.cs.cmu.edu/~410-s05/lectures/L31\\_LockFree.pdf](https://www.cs.cmu.edu/~410-s05/lectures/L31_LockFree.pdf).
- Also be sure to check out this presentation by Samy Al Bahra for a clear and easily digestible overview of pretty much every topic under the sun related to concurrent programming: [http://concurrencykit.org/presentations/lockfree\\_introduction/#/](http://concurrencykit.org/presentations/lockfree_introduction/#/).
- Mike Acton's excellent talk on concurrent thinking is a must-read; it is available at [http://cellperformance.beyond3d.com/articles/public/concurrency\\_rabit\\_hole.pdf](http://cellperformance.beyond3d.com/articles/public/concurrency_rabit_hole.pdf).
- These two online books are great resources for concurrent programming: <http://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf> and <https://www.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.2011.01.02a.pdf>.
- Some excellent articles on lock-free programming and how atomics, barriers and fences work can be found on Jeff Preshing's blog: <http://preshing.com/20120612/an-introduction-to-lock-free-programming>.